

Assignment 2A SQL and R LLM Transcript – Brandon Chanderban

The following outlines the interactions I had with the LLM during the completion of this assignment, arranged in the order they occurred. Questions are denoted in green.

Q. How do I go about linking created tables (users, movies, and ratings) within PostgreSQL (pgAdmin 4) to my RStudio project?

To **link tables created in PostgreSQL (via pgAdmin 4)** to your **RStudio project**, you'll follow these steps:

✅ Step-by-Step: Connecting PostgreSQL to RStudio

1. Ensure Required Packages Are Installed

In RStudio, install the following packages:

```
install.packages("DBI")
```

```
install.packages("RPostgres")
```

Then load them:

```
library(DBI)
```

```
library(RPostgres)
```

2. Create a Connection to Your Database

Use `dbConnect()` to link to your PostgreSQL database. Example:

```
con <- dbConnect(  
  RPostgres::Postgres(),  
  dbname = "movie_ratings", # Your database name  
  host = "localhost",      # Or IP address if remote  
  port = 5432,             # Default PostgreSQL port  
  user = "postgres",       # Your PostgreSQL username  
  password = "your_password" # Password (use env var if possible)  
)
```

🔒 Optional: Secure the password

Store it in `.Renviron` (in your project folder):

```
DB_PASSWORD=your_password_here
```

Then restart RStudio and use:

```
password = Sys.getenv("DB_PASSWORD")
```

This keeps your password **out of version control** (especially important for GitHub).

3. Test the Connection

Check available tables:

```
dbListTables(con)
```

4. Load Your Tables into R as Data Frames

Assuming you have tables called users, movies, and ratings:

```
users <- dbGetQuery(con, "SELECT * FROM users")
```

```
movies <- dbGetQuery(con, "SELECT * FROM movies")
```

```
ratings <- dbGetQuery(con, "SELECT * FROM ratings")
```

These are now **R data frames** you can work with using dplyr, ggplot2, etc.

5. Always Disconnect When Done

```
dbDisconnect(con)
```

Done!

You've now **linked your PostgreSQL database (created in pgAdmin 4) to RStudio**, and pulled your tables for analysis.

Let me know if you'd like this turned into a reusable code template!

Q: How do I join the three tables together in R to get full context on ratings?

Use left_join() twice:

```
ratings_full <- ratings %>%
```

```
  left_join(users, by = "user_id") %>%
```

```
  left_join(movies, by = "movie_id")
```

This gives you a single dataframe with rating, user name, and movie title.

Q: How can I calculate average rating per movie, and count how many users rated it? (Explain each step following the code)

You can calculate both the average rating per movie and the number of users who rated each movie by grouping the data by movie title and summarizing the ratings. An example using dplyr is shown below:

```
ratings_full %>%
```

```
  group_by(title) %>%
```

```
  summarise(
```

```
    avg_rating = mean(rating, na.rm = TRUE),
```

```
    num_ratings = sum(!is.na(rating))
```

```
  ) %>%
```

```
  arrange(desc(avg_rating))
```

Explanation of each step:

```
ratings_full %>%
```

This begins a pipeline using the pipe operator, which passes the dataset (ratings_full) through each subsequent function in sequence.

```
group_by(title)
```

This groups the data by movie title so that all calculations that follow are performed separately for each movie.

```
summarise(...)
```

This reduces each group (each movie) down to summary statistics:

```
avg_rating = mean(rating, na.rm = TRUE)
```

 calculates the average rating for each movie while ignoring missing values (NA).

```
num_ratings = sum(!is.na(rating))
```

 counts how many users actually rated each movie by counting non-missing ratings.

```
arrange(desc(avg_rating))
```

This sorts the results in descending order of average rating, making it easy to identify the highest- and lowest-rated movies.

Together, this code produces a concise summary table showing how well each movie was rated on average and how many users contributed ratings, allowing for more informed interpretation of the results.

Q: Can you assist with the R code for generating a bar plot using ggplot2? I want the average reviews per movie to be placed upon the vertical axis, with the horizontal one being used to visualize the averages. The counts of reviews per movie can be placed as a label, at the end of each bar. Explain the code afterwards.

How can I generate a bar plot of average movie ratings with number of reviews labeled?

```
library(ggplot2)
```

```
# Summarize data first
```

```
ratings_summary <- ratings_full %>%
```

```
  group_by(title) %>%
```

```
  summarise(
```

```

avg_rating = mean(rating, na.rm = TRUE),
num_ratings = sum(!is.na(rating))
) %>%
arrange(desc(avg_rating))

# Create horizontal bar plot
ggplot(ratings_summary, aes(x = reorder(title, avg_rating), y = avg_rating)) +
  geom_col(fill = "steelblue") +
  geom_text(aes(label = num_ratings), hjust = -0.1, size = 3.5) +
  coord_flip() +
  labs(
    title = "Average Movie Ratings",
    subtitle = "Labeled with Number of Ratings per Movie",
    x = "Movie Title",
    y = "Average Rating"
  ) +
  ylim(0, 5.5) +
  theme_minimal()

```

Explanation of Each Step:

- **ratings_summary <- ratings_full %>% ...**
Before plotting, the data is summarized to compute:
 - avg_rating: the average rating for each movie (ignoring missing ratings).
 - num_ratings: the total number of ratings submitted for each movie.
- **ggplot(ratings_summary, aes(...))**
Initializes the ggplot with the summarized data and sets up the axes:
 - x = reorder(title, avg_rating): orders the movie titles by their average rating (low to high).
 - y = avg_rating: sets the vertical axis as the average rating.
- **geom_col(fill = "steelblue")**
Creates the bars using the average ratings. A solid fill color is applied for clarity.
- **geom_text(aes(label = num_ratings), hjust = -0.1)**
Adds numeric labels to each bar showing how many people rated the movie. The hjust = -0.1 positions the text just outside the bar for visibility.
- **coord_flip()**
Flips the chart to make it a **horizontal bar chart**, which is easier to read when movie titles are long.
- **labs(...)**
Adds a clear title, subtitle, and axis labels.

- **ylim(0, 5.5)**
Sets the y-axis range to give space for the rating scale (1 to 5) and leaves room above the tallest bar for the labels.
- **theme_minimal()**
Applies a clean and professional style.